

Introduzione

La **programmazione dinamica** è una tecnica basata sulle iterazioni (*e non ricorsioni*). Similmente al ***divide et impera*** anche questa tecnica lavora su dei *sotto-problemi*, ma la *programmazione dinamica* conserva ogni soluzione di questi sotto-problemi all'interno di una tabella, così da non ripetersi, a differenza della tecnica gemella; l'uso di questa tabella evita quindi calcoli ridondanti.

Tra le applicazioni più conosciute della tecnica abbiamo: **sottovettore di somma massima**, ***edit distance***, ***Bellman-Ford*** e tanti altri.

Edit Distance (*Levenshtein distance*)

L'algoritmo misura i **passaggi minimi** per trasformare una stringa X in una stringa Y .

Ogni carattere della prima stringa viene confrontato con un carattere della seconda, e a ogni *operazione* viene associato un **costo**, quest'ultimo verrà inserito nella posizione $[x_i, y_i]$ di una matrice ausiliaria D , che conterrà ogni costo di ogni operazione.

La parte **iterativa** di questo algoritmo si può sintetizzare in questo modo:

All'iterazione (i, j) vengono confrontati i caratteri $x_i = y_i$, se uguali il costo sarà identico al passaggio precedente $D[i, j] = D[i - 1, j - 1]$, ovvero la "cella in alto a sinistra".

Se invece $x_i \neq y_i$ si valuta che operazione applicare: